Faculté des sciences de Monastir
Niveau LSI3

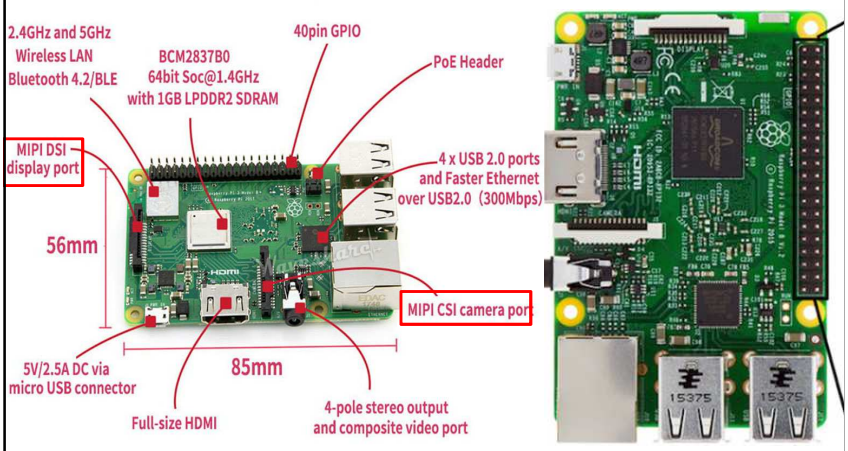
Année Universitaire 2025-26

Applications IOT via Raspberry Pi



Proposé par Mr Samir Belaid

Initiation Raspberry Pi



Raspberry Pi 3 GPIO Header

Pin#	NAME		NAME	Pin#
01	3.3v DC Power		DC Power 5v	02
03	GPIO02 (SDA1 , I²C)		DC Power 5v	04
05	GPIO03 (SCL1 , I²C)		Ground	06
07	GPIO04 (GPIO_GCLK)		(TXD0) GPIO14	08
09	Ground		(RXD0) GPIO15	10
11	GPIO17 (GPIO_GEN0)		(GPIO_GEN1) GPIO18	12
13	GPIO27 (GPIO_GEN2)		Ground	14
15	GPIO22 (GPIO_GEN3)		(GPIO_GEN4) GPIO23	16
17	3.3v DC Power		(GPIO_GEN5) GPIO24	18
19	GPIO10 (SPI_MOSI)		Ground	20
21	GPIO09 (SPI_MISO)		(GPIO_GEN6) GPIO25	22
23	GPIO11 (SPI_CLK)		(SPI_CE0_N) GPIO08	24
25	Ground		(SPI_CE1_N) GPIO07	26
27	ID_SD (I²C ID EEPROM)		(I²C ID EEPROM) ID_SC	28
29	GPIO05		Ground	30
31	GPIO06		GPIO12	32
33	GPIO13		Ground	34
35	GPIO19		GPIO16	36
37	GPIO26		GPIO20	38
39	Ground		GPIO21	40

Rev. 2
29/02/2016

www.element14.com/RaspberryPi

Bien débuter sur le Raspberry Pi

- Le *Raspberry Pi* charge son système d'exploitation à partir du lecteur de carte micro SD.
- *Raspberry Pi OS* (https://www.raspberrypi.org/software/) est un logiciel simplifiant le processus d'installation du système d'exploitation.
- Cliquer sur « *See all download options* »; deux configurations existent : *OS with desktop* ou *OS Lite* (une image *avec* ou *sans* interface *graphique*)
- Pour installer *OS*, il suffit de copier les fichiers décompressés sur la carte SD formatée en *FAT32*.(avec ***Balena Etcher*** *pour la partition/validation*)
- Le premier démarrage va nécessiter un *écran* avec *entrée HDMI*, un *clavier* ainsi qu'une *souris*.
- Le Raspberry Pi s'initialise... puis le *bureau apparaît* après quelques secondes.

Se connecter au bureau de son Raspberry depuis un PC Windows

Configurations sur Raspberry

Du côté du Raspberry, il suffit d'installer les outils de bureau à distance pour Linux : *xrdp* et *tightvncserver*

```
sudo apt-get install tightvncserver
sudo apt-get install xrdp
```

Configurations sur Windows

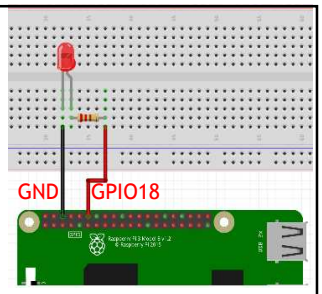
- Lancer l'outil **"Connexion Bureau à distance"**,
- Une fois ouvert, entrez l'adresse IP du Raspberry (disponible en tapant *ifconfig* dans un terminal),
- Puis, connectez vous avec vos identifiants et mots de passe (par défaut ***pi*** et ***raspberry***).

Autres outils utiles

D'autres outils sont utiles lors de l'utilisation, par exemple, de l'appareil photo ou d'une webcam:

- ***SSH***: Utilisée pour accéder à distance au *Raspberry Pi* via un réseau local
- ***SCP***: permette de copier via *SSH* des photos prises par *Pi* sur un ordinateur principal
- ***rsync***: Utilisée pour synchroniser les dossiers des photos prises entre Pi et l'ordinateur
- **cron**: Planifie la prise d'une photo à un intervalle donné, comme chaque minute pour capturer un laps de temps

Créer et exécuter un programme Python



Branchement d'une LED au Raspberry

- Pour relier la LED au Raspberry Pi, on a besoin d'une résistance (en général 100Ω).

Création du fichier Python *.py*

- Une fois que tout est branché, vous pouvez vous connecter avec un clavier/souris ou par ***SSH*** pour écrire sur le *Raspberry Pi* votre premier programme en Python.
- *Avec Raspberry Pi* les commandes entrées sont des commandes *Linux*.
- On peut par exemple appeler le programme comme suit :

```
sudo nano led.py
```

Cette commande crée le fichier *led.py* dans le répertoire principal (***/home/pi*** par défaut) et l'ouvre dans un éditeur de texte.

Ecriture du programme python

```
import RPi.GPIO as GPIO # Importation des librairies qui gèrent les ports
import time # Importation de la librairie temps
GPIO.setmode(GPIO.BCM) # BCM : Numéro des GPIO (GPIO 18)
GPIO.setup(18, GPIO.OUT) # Définition du port en sortie
GPIO.setwarnings(False) # Mettre sur OFF les alertes (qui sont inutiles)
# Affichage de texte
print("\n+---------------------/ Blink LED /-------------------------+")
print("| La LED doit etre reliée au GPIO 18 du Raspberry |")
print("+-----------------------------------------------------------------+\n")
nbrBlink = int(input("Combien de fois la LED doit clignoter ?\n")) # Utilisation de la fonction input pour acquérir des informations
tempsAllume = int(input("Combien de temps doit-elle rester allumée ?\n"))
tempsEteint = int(input("Combien de temps doit-elle rester éteinte ?\n"))
i = 0 # Définition d'une variable type compteur
while i < nbrBlink :
    GPIO.output(18, True) # Mise a 1 du GPIO 18 (+5V)
    time.sleep(tempsAllume) # On attend le temps défini
    GPIO.output(18, False) # Mise a zero du GPIO 18 (GND)
    time.sleep(tempsEteint) # ...
    i = i+1
GPIO.cleanup()
```

Exécution du programme Python

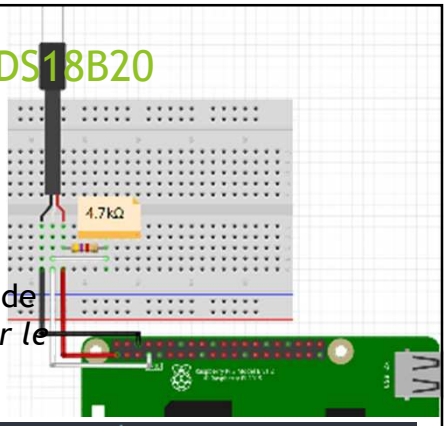
Pour lancer le programme que vous avez créé, il suffit d'entrer :

```
sudo python led.py
```

Lire la température à l'aide d'une sonde DS18B20

Branchements de la sonde

On branche la sonde au *Raspberry* en suivant le schéma ci-devant.



Lecture via la commande du Raspberry

On peut lire la sonde en utilisant l'invite de commande du *Raspberry* comme ceci : *(en remplaçant les 'x' par le numéro donné par le Raspberry).*

```
sudo modprobe w1-gpio
sudo modprobe w1-therm
cd /sys/bus/w1/devices/
ls
cd 28-xxxxxxxxxx
cat w1_slave
```

```
pi@EspaceRaspberry:/sys/bus/w1/devices $ sudo modprobe w1-gpio
pi@EspaceRaspberry:/sys/bus/w1/devices $ sudo modprobe w1-therm
pi@EspaceRaspberry:/sys/bus/w1/devices $ cd /sys/bus/w1/devices/
pi@EspaceRaspberry:/sys/bus/w1/devices $ ls
28-0317229e0bff  w1_bus_master1
pi@EspaceRaspberry:/sys/bus/w1/devices $ cd 28-0317229e0bff
pi@EspaceRaspberry:/sys/bus/w1/devices/28-0317229e0bff $ cat w1_slave
6e 00 4b 46 7f ff 0c 10 90 : crc=90 YES
6e 00 4b 46 7f ff 0c 10 90 t=6875
pi@EspaceRaspberry:/sys/bus/w1/devices/28-0317229e0bff $
```

Sonde DS18B20 non détectée ?

Il peut arriver que le *Raspberry* ne détecte pas la sonde de température.
Pour qu'il la détecte, il faut modifier deux fichiers système :

Ajouter les lignes suivantes au ficher qui se trouve dans */etc/modules* :

```
sudo nano /etc/modules
```

```
w1-therm
w1-gpio pullup=1
```

Ajouter la ligne suivante au fichier qui se trouve dans */boot/config.txt* :

```
dtoverlay=w1-gpio
```

Redémarrer le *Raspberry Pi*.

Afficher la température avec un script Python

On peut créer un programme qui affiche la température au millième.

```
def lireFichier (emplacement) :
    # Ouverture du fichier contenant la temperature
    fichTemp = open(emplacement)
    # Lecture du fichier
    contenu = fichTemp.read()
    # Fermeture du fichier apres qu'il ai ete lu
    fichTemp.close()
    return contenu

def recupTemp (contenuFich) :
    # Supprimer la premiere ligne qui est inutile
    secondeLigne = contenuFich.split("\n")[1]
    temperatureData = secondeLigne.split(" ")[9]
    # Supprimer le "t="
    temperature = float(temperatureData[2:])
    # Mettre un chiffre apres la virgule
    temperature = temperature / 1000
    return temperature

contenuFich = lireFichier("/sys/bus/w1/devices/28-xxxxxxxx/w1_slave")
temperature = recupTemp (contenuFich)

print ("Temperature :", temperature)
```

On obtient, en lançant le programme python ceci:

```
pi@EspaceRaspberry:~ $ sudo python3 temp.py
Temperature :16.187
pi@EspaceRaspberry:~ $
```

Enregistrer les températures dans un fichier texte

- On ajoute une ligne dans un fichier à chaque prise de température.
- Pour changer l'intervalle de lecture, modifier le *time.sleep()* qui est en seconde.

```
import time
import datetime

def lireFichier (emplacement) :
    fichTemp = open(emplacement)
    contenu = fichTemp.read()
    fichTemp.close()
    return contenu

def recupTemp (contenuFich) :
    secondeLigne = contenuFich.split("\n")[1]
    temperatureData = secondeLigne.split(" ")[9]
    temperature = float(temperatureData[2:])
    temperature = temperature / 1000
    return temperature

def sauvegarde (temperature, date, emplacement) :
    fichierSauvegarde = open(emplacement, "a")
    fichierSauvegarde.write(str(date)+"   ")
    fichierSauvegarde.write(str(temperature)+'\r\n')
    fichierSauvegarde.close()

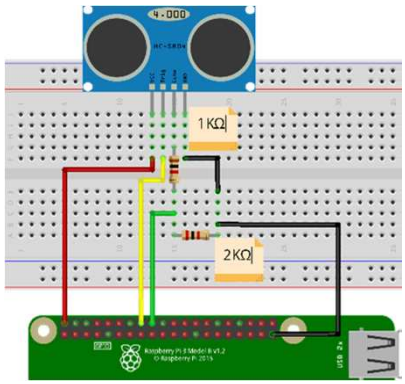
while True :
    date = datetime.datetime.now().strftime('%Y-%m-%d %H:%M:%S')
    contenuFich = lireFichier("/sys/bus/w1/devices/28-xxxxxxxx/w1_slave")
    temperature = recupTemp(contenuFich)
    sauvegarde(temperature, date, "Temperature.txt")
    time.sleep(60)
```

Mesure de distances avec un capteur HC-SR04

Le capteur à ultrasons HC-SR04 permet de mesurer une distance de 2 à 400cm en envoyant des signaux sonores à 40 kHz. En fonction du **temps** qui sépare **l'émission** de la **réception**, on trouve par un calcul la **distance**.

Branchement du capteur au Raspberry

- Le HC-SR04 se connecte grâce à 4 PINs: *VCC*, *Trig*, *Echo* et *GND*. Pour le connecter, on a besoin de deux résistances : *1KΩ et 2KΩ*.



- Une fois branché, il suffit d'exécuter un script Python pour obtenir la distance calculée entre le capteur et l'obstacle.

Programmation du capteur en Python

Ce script va donner une distance en centimètres toutes les secondes pendant un nombre de fois donné :

```
import RPi.GPIO as GPIO
import time
GPIO.setmode(GPIO.BCM)
print "+---------------------------------------------------------+"
print "|Mesure de distance par le capteur ultrasonore HC-SR04|"
print "+---------------------------------------------------------+"
Trig = 23          # Entree Trig du HC-SR04 branchee au GPIO 23
Echo = 24         # Sortie Echo du HC-SR04 branchee au GPIO 24
GPIO.setup(Trig,GPIO.OUT)
GPIO.setup(Echo,GPIO.IN)
GPIO.output(Trig, False)
repet = input("Entrez un nombre de repetitions de mesure : ")
for x in range(repet):    # On prend la mesure "repet" fois
   time.sleep(1)       # On la prend toute les 1 seconde
   GPIO.output(Trig, True)
   time.sleep(0.00001)
   GPIO.output(Trig, False)
   while GPIO.input(Echo)==0:  ## Emission de l'ultrason
     debutImpulsion = time.time()
   while GPIO.input(Echo)==1:   ## Retour de l'Echo
     finImpulsion = time.time()
   distance = round((finImpulsion-
debutImpulsion)*340*100/2,1)## Vitesse du son=340m/s
   print "La distance est de : ",distance," cm"
GPIO.cleanup()
```

L'exécution de ce programme donne :

```
pi@EspaceRaspberryFrancais:~ $ sudo python distance.py
+---------------------------------------------------------+
|   Mesure de distance par le capteur ultrasonore HC-SR04   |
+---------------------------------------------------------+
Entrez un nombre de repetitions de mesure : 15
La distance est de :  76.3  cm
La distance est de :  77.9  cm
```

Communication USB entre Arduino UNO et Raspberry Pi

Pour tirer l'avantage de chaque carte, le mieux est d'établir une communication entre les deux. Nous allons voir comment mettre en place les 3 types de communication suivants :

1. Communication unidirectionnelle Arduino UNO → Raspberry Pi,
2. Communication unidirectionnelle Raspberry Pi → Arduino UNO,
3. Communication bidirectionnelle Raspberry Pi ↔ Arduino UNO : mélange les deux précédentes et permet à la fois d'envoyer et recevoir des données.

Communication unidirectionnelle de l'Arduino vers le Raspberry Pi

- Côté ***Arduino***, il suffit de lui demander d'écrire sur la sortie série *Serial* le message que l'on souhaite transmettre.
- Ne branchez pas encore l'Arduino au Raspberry Pi.

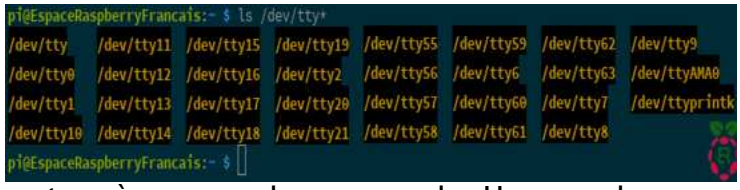
```
int i=0;
void setup() {
  Serial.begin(9600);
  Serial.println("Un message va etre envoye
toutes les deux secondes des maintenant!");}
void loop() {
  Serial.println("Message #" + String(i));
  delay(2000);
  i++;}
```

Communication unidirectionnelle de l'Arduino vers le Raspberry Pi

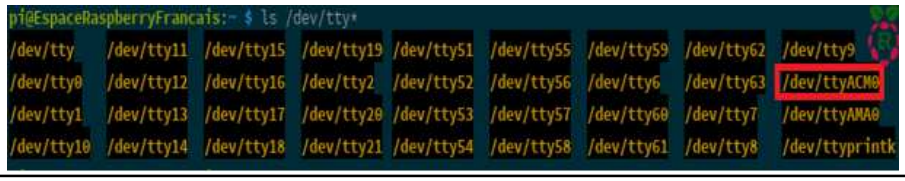
Côté *Raspberry*, on écrit un programme en Python qui utilise la fonction *serial.Serial*. Cependant, on a besoin pour utiliser cette fonction, du port utilisé par l'Arduino sur le Raspberry Pi.
Il suffit d'entrer la commande Linux suivante :

```
ls /dev/tty*
```

On obtient une liste de ce type :



Brancher l'Arduino au port USB et entrez à nouveau la commande. Un nouvel élément devrait apparaître dans la liste (c'est le nom que prend l'Arduino).

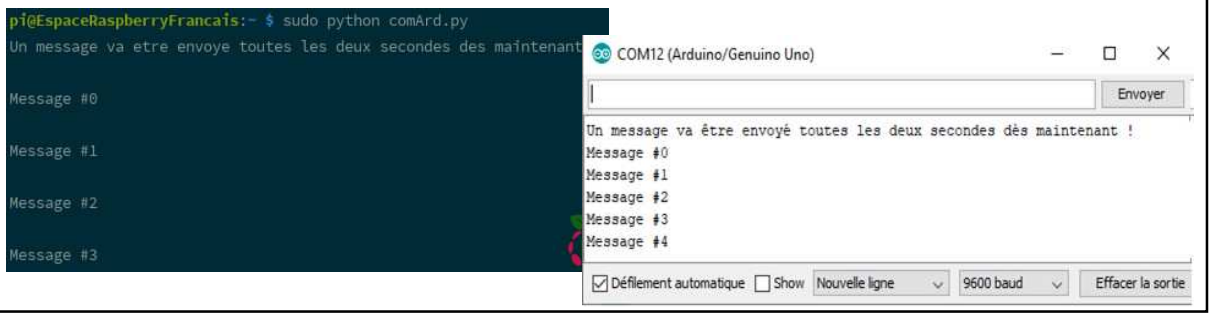


Communication unidirectionnelle de l'Arduino vers le Raspberry Pi

Dans ce cas, l'Arduino est détecté en temps que */dev/ttyACM0*.
Regardons maintenant le programme qui sera exécuté sur le *Raspberry* :

```
# Import de la librairie serial
import serial
# Ouverture du port série avec :
# '/dev/ttyXXXX' : définition du port d écoute (remplacer 'X' par le bon nom)
# 9600 : vitesse de communication
serialArduino = serial.Serial('/dev/ttyXXXX', 9600)
# Ecriture de chaque message reçu
while True :
    print(serialArduino.readline())
```

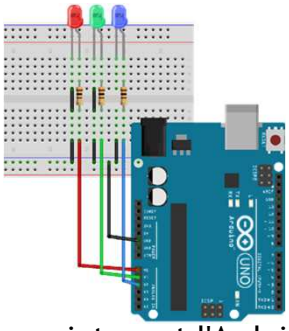
On obtient à l'exécution ceci :



Communication unidirectionnelle du Raspberry Pi vers l'Arduino

Pour cette communication, On va contrôler des LED connectées à l'Arduino à partir du Raspberry.

- Commencer par brancher les LEDs à l'Arduino en suivant le schéma ci-dessous:



- Programmer maintenant l'Arduino, qui va recevoir les ordres du Raspberry Pi:

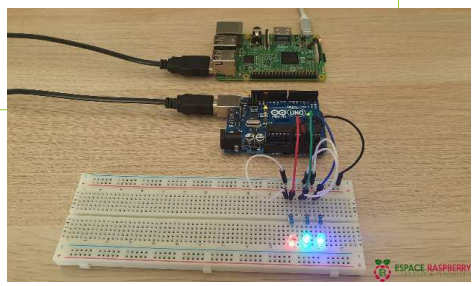
```
#define Rouge A0
#define Vert A1
#define Bleu A2
 int message = 0;
int couleurs[] = {Rouge, Vert, Bleu};
void setup() {
  Serial.begin(9600);
  pinMode(Rouge, OUTPUT);
  pinMode(Vert, OUTPUT);
  pinMode(Bleu, OUTPUT);}
 void loop() {
  if (Serial.available())  {
    message = Serial.read() - '0';
    if (message > 3) {
  digitalWrite(couleurs[message-4], LOW);}
  else { digitalWrite(couleurs[message-1], HIGH);}
  } }
```

Communication unidirectionnelle du Raspberry Pi vers l'Arduino

Pour finir, on code l'envoi des ordres par le Raspberry Pi :

```
import serial
ser = serial.Serial('/dev/ttyXXXX', 9600)
print("CTRL + C pour arrêter")

while True :
    led = int(input('Quelle LED souhaitez-vous allumer / éteindre ?
(Rouge=1 / Vert=2 / Bleu=3)'))
    action = input("Souhaitez-vous allumer ou Eteindre la LED ?
(Allumer=1 / Eteindre=0)")
    if (action == 1) :
        ser.write(str(led))
    else :
        msg = led+3
        ser.write(str(msg))
```



Débuter avec Node-Red sur Raspberry Pi

- ***Node-Red*** est un langage de programmation graphique pour développer des objets connectés, idéal pour comprendre l'IOT,
- Disponible sur Windows, macOS et inclus par défaut la distribution **Raspbian**.

Démarrer Node-Red

- Pour démarrer ***Node-Red*** sur Raspberry dans Menu -> Programmation
- On peut également démarrer depuis le Terminal avec la commande:

```
sudo node-red-start
```

- Le serveur web est maintenant démarré et accessible depuis le réseau local.
- L'utilisation du ***Node-Red*** peut se faire, depuis n'importe quel ordinateur, dans un navigateur internet en saisissant:

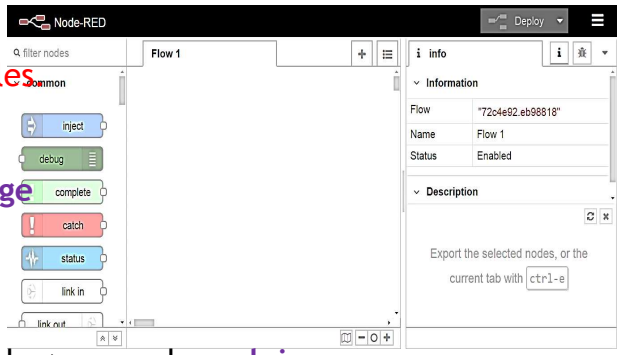
http://ADRESSE_IP_DU_RASPBERRY:1880/

- Si on veut programmer directement sur votre Pi, ouvrir le navigateur internet et saisir: ***http://localhost:1880/*** ou ***http://127.0.0.1:1880/***

```
sudo node-red-stop  #pour arrêter NodeRed
```

Découvrir l'environnement de programmation

- Au **centre** la zone de **programmation.**
- À **gauche** toutes les **fonctions** disponibles.
- À **droite** les onglets informations contextuelles et **debug.**
- **Au coin supérieur droit,** le bouton **rouge Deploy,** pour **déployer le programme.**



Installer (ou mettre à jour) npm

- **Node-Red** utilise le gestionnaire de package **npm** de **nodejs**.

```
sudo apt-get install npm
sudo npm i npm -g  #on met à jour npm
```

Installation de *node-red-admin*

```
cd /home/pi/.node-red  # aller dans le répertoire caché de node-red
npm install -g node-red-admin
```

Faire clignoter une Led avec Node-Red

- Pour faire clignoter une ***Led*** reliée sur le ***GPIO23*** du ***Raspberry Pi*** on utilise la fonction **inject** qui envoi une pulsation, chaque seconde (au choix). A chaque pulsation, on utilise la fonction "***Inverse Etat Led***" pour allumer ou éteindre la ***Led***.



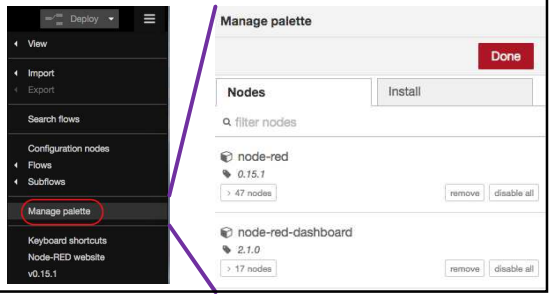
- ***Copier*** puis ***Importer*** directement ce code dans le **flow** (***Menu->Import->Clipboard***).


```
[{"id":"3ff9c2cd.ef914e","type":"function","z":"16966388.6f0e2c","name":"Inverse état Led","func":"\ncontext.state = context.state || 0;\n\n(context.state == 0) ? context.state = 1 : context.state = 0;\nmsg.payload = context.state;\n\nreturn msg;","outputs":1,"noerr":0,"x":340,"y":320,"wires":[["b91a94fa.ec1e48","59b95cea.56d4f4"]]},{"id":"b91a94fa.ec1e48","type":"debug","z":"16966388.6f0e2c","name":"","active":true,"x":570,"y":360,"wires":[]},{"id":"9b2414e2.c584c8","type":"inject","z":"16966388.6f0e2c","name":"chaque seconde","topic":"","payload":"","payloadType":"date","repeat":"1","crontab":"","once":false,"x":149,"y":320.0000009536743,"wires":[["3ff9c2cd.ef914e"]]},{"id":"59b95cea.56d4f4","type":"rpi-gpio out","z":"16966388.6f0e2c","name":"Led sur GPIO23","pin":"16","set":false,"out":"out","x":580,"y":280,"wires":[[]]}]
```


Installer le module *node-red-node-openweathermap*

- La communauté **Node-Red** développe de très nombreux plugins dans tous les domaines :
  - Objets connectés (ZWave, Sonos..),
  - Services en ligne (météo, twitter…),
  - Bases de données...
- On propose installer le module ***node-red-node-openweathermap*** qui permet de récupérer la météo locale depuis le site ***openweathermap***.
- Ouvrir le gestionnaire de palette, chercher le module ***node-red-node-openweathermap*** puis cliquez sur ***install***.
- En cas de **difficulté** pour installer le module, ***arrêter NR*** puis **exécuter** la commande dans le Terminal.

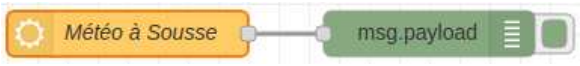
```
npm install node-red-node-openweathermap
```

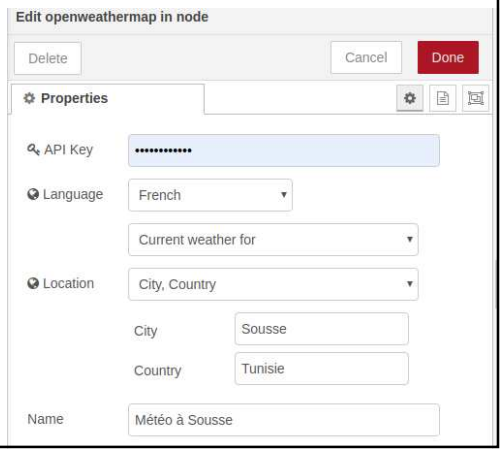


Récupérer la météo locale depuis OpenWeatherMap

- Pour récupérer la météo, on doit récupérer une ***clé API*** qui est gratuite pour un usage personnel (voir le lien https://openweathermap.org/).
- Pour récupérer la clé, créer un compte, puis aller dans l'onglet **API keys** de **My profile**
- Ajouter un node ***openweathermap*** (celui du bas) dans un flow. Faites un double clic puis :

  - Coller la clé API
  - Indiquer la ville (et le pays) ou les coordonnées GPS du lieu dont on souhaite connaître la météo
  - Brancher un ***node debug*** pour envoyer la météo récupérée dans la console



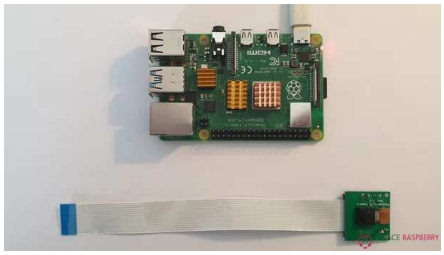


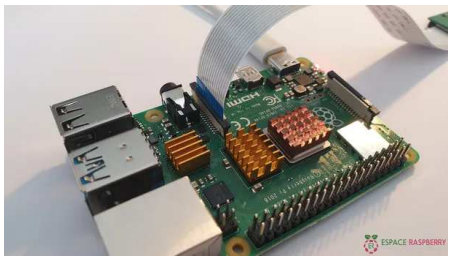
Utiliser une caméra sur un Raspberry Pi: Prendre des photos et vidéos

- En plus de tous les composants que l'on peut relier aux GPIOs, le Raspberry Pi possède deux connecteurs particuliers : les ports **DSI** - *Display Serial Interface* et **CSI** - *Camera Serial Interface*. Ils permettent respectivement de relier un écran et une camera au Raspberry Pi.

Branchement de la caméra

- La caméra se branche sur le bus CSI du Raspberry pi, spécialement conçu pour les caméras et qui offre un **débit élevé**.



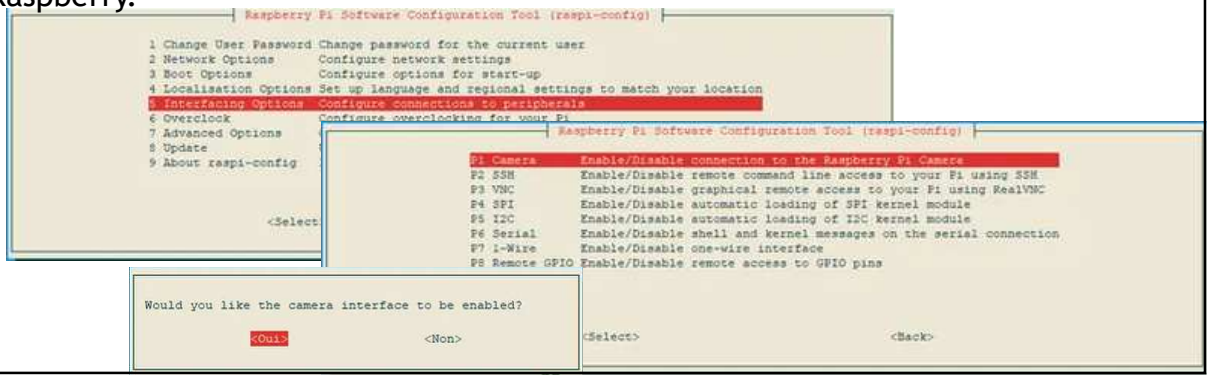


Préparation du Raspberry

Dans un premier temps, il faut activer l'interface de la caméra. Pour se faire, entrez la commande suivante dans le terminal.

```
sudo raspi-config
```

Une fois dans le menu, sélectionner ***Interfacing Options*** puis dans ***Camera***. Puis, sélectionnez ***Yes*** pour confirmer l'activation ainsi que pour redémarrer le Raspberry.



Détection de la caméra

Avant de continuer, voici une commande qui permet de vérifier que la caméra soit bien détectée :

```
vcgencmd get_camera
```

Si la ligne affichée n'est pas *supported*=1 *detected*=1, vérifier :

- Que la caméra soit branchée correctement,
- Que l'interface camera est bien activée.

Si *detected* n'est toujours pas à 1 après ces vérifications, redémarrer le Raspberry Pi et essayer de le mettre à jour :

```
sudo apt-get update
sudo apt-get upgrade
```

La caméra devrait maintenant être détectée.

Prendre une photo

A. Par le Terminal

La première chose que l'on va faire avec cette caméra est de prendre des photos. Pour ce faire, entrer les commandes suivantes :

```
cd
mkdir Camera
raspistill -o ~/Camera/image.jpg -q 100
```

La commande de la 3ième ligne prend une photo et la stocke dans le dossier créé.
La commande *raspistill* accepte plusieurs arguments (*raspistill --help*) dont :

-o : output filename - Endroit ou va être stockée la photo
-q : quality - 0 à 100, Qualité de l'image
-w et -h : width et height - Permet de définir manuellement les dimensions de l'image
-t : timeout - Temps mis avant de prendre la photo (retardateur)
-vf et -hf : vertical flip et horizontal flip - Retourne la photo

Prendre une photo

B. En Python

Il est beaucoup plus pratique pour des éventuels projets de pouvoir prendre des photos directement à l'intérieur d'un script python. Pour se faire, il existe la librairie *picamera*. Cette librairie est normalement installée de base sur le *Raspberry* mais rien n'enpèche de vérifier. Lançons donc la commande :

```
sudo apt-get install python3-picamera
```

Cette fois, la librairie est forcément installée. Commençons par créer un simple script Python qui prend et sauvegarde une photo. (en ayant comme nom la date)

```
1  import picamera
2  import datetime
3  camera = picamera.PiCamera()
4  camera.resolution = (2592,1944)
5  camera.capture(str(datetime.datetime.now())+'.jpg')
```

Prendre une photo

Définir la résolution ?

La ligne *camera.resolution* n'est pas obligatoire. Cependant, si elle n'est pas renseignée, la résolution choisie sera de 1920x1080, et la qualité de la photo sera très mauvaise (surtout en basse luminosité). Si on souhaite une photo de la meilleure qualité possible:

- *camera.resolution* = (2592, 1944) si on dispose d'une camera 5 MP,
- *camera.resolution* = (3280, 2464) si on dispose d'une camera 8 MP.

Photo à l'envers ?

Tourner une photo: En fonction de comment est positionné votre caméra, l'image capturée peut être à l'envers. Pour corriger ceci, ajouter ceci après la ligne "*camera=PiCamera()*" :

```
camera.hflip = True
camera.vflip = True
```

Prendre une photo

Photo à l'envers ?

Ce qui donne, le code suivant:

```
1 import picamera
2 import datetime

3 camera = picamera.PiCamera()
4 camera.hflip = True
5 camera.vflip = True
6 camera.resolution = (2592,1944)

7 camera.capture(str(datetime.datetime.now())+'.jpg')
```

D'autres options pouvant s'ajouter au même endroit sont disponibles pour les plus photophyles dont voici une liste :

```
1  camera.sharpness = 0
2  camera.contrast = 0
3  camera.brightness = 50
4  camera.saturation = 0
5  camera.ISO = 0
6  camera.video_stabilization = False
7  camera.exposure_compensation = 0
8  camera.exposure_mode = 'auto'
9  camera.meter_mode = 'average'
10 camera.awb_mode = 'auto'
11 camera.image_effect = 'none'
12 camera.color_effects = None
13 camera.rotation = 0
14 camera.crop = (0.0, 0.0, 1.0, 1.0)
```

Prendre une vidéo

A. Par le Terminal

Pour enregistrer une vidéo de 10 secondes (10 000 ms) sous le nom de fichier *video.h264*, il suffit d'entrer la commande suivante.

```
raspivid -o video.h264 -t 10000
```

Les options de cette commande sont les mêmes que pour la photo (comme hf et vf par exemple).

Problème: la plupart des lecteurs vidéo ne sauront pas lire le format **.h264**, Pour cette raison, on va installer la librairie **MP4Box** qui va nous permettre de convertir les vidéos au format **MP4**. Pour installer **MP4Box**, il suffit d'entrer cette commande :

```
sudo apt-get install -y gpac
```

Une fois le logiciel installé, exécutez ceci afin de convertir le fichier:

```
MP4Box -add video.h264 video.mp4
```

Le fichier vidéo est maintenant lisible sur tous les appareils.

Prendre une vidéo

B. En Python

Voici un script qui permet de capturer une vidéo en Python et de la convertir en MP4 automatiquement:

```
1  import os
2  import picamera
3  import time

4  camera = picamera.PiCamera()
5  #Retourner image : Enlever les deux hashtag  suivants
6  #camera.hflip = True
7  #camera.vflip = True
8  dureeVideo = 5
9  nomVideo = time.strftime("%d.%m.%Y-%Hh%Mm%Ss")
10 camera.start_recording(nomVideo+".h264")
11 print("REC...")
12 time.sleep(dureeVideo)
13 camera.stop_recording()
14 commande = "MP4Box -add "+nomVideo+".h264 " + nomVideo+".mp4"
15 os.system(commande)
```

Si on souhaite que la Led de la caméra reste éteinte, il suffit de modifier le fichier ***/boot/config.txt*** en ajoutant la ligne ***disable_camera_led=1*** à la fin.

Faire du streaming

Pour un système de vidéo surveillance ou pour le pilotage d'un robot à distance, il faut pouvoir récupérer le flux vidéo et le diffuser à travers le réseau (streaming).
Il y a 5 façons pour faire du streaming:

Méthode 1 : Netcat et Mplayer

- Il s'agit de rediriger le flux capturé par la commande **raspivid** dans **netcat**,
- Sur un PC sous Linux exécuter la commande suivante (installer **mplayer**),

```
nc -l -p 5001 | /usr/bin/mplayer -fps 10 -cache 1024 -
```

- Cette commande ordonne à **nc** (*netcat*) d'écouter sur le **port 5001** et redirige ce qui arrive dans le lecteur vidéo **Mplayer** (**1024 Ko** de **cache** pour **10 images/s**).
- Sur le Pi, exécuter **raspivid** et rediriger la sortie vers l'adresse **IP** du PC client en utilisant **netcat** :

```
raspivid -t 0 -w 1280 -h 720 -o - | nc adresse_ip_du_pc_client 5001
```

- Pour enregistrer le flux vidéo dans un fichier sur le PC client, modifier la première commande:

```
nc -l -p 5001 | /usr/bin/mplayer -fps 10 -cache 1024 -demuxer h264es -dumpstream -dumpfile
./Bureau/ma_video.h264 -
```

Faire du streaming

Méthode 2 : streaming VLC

- Avec cette méthode, le flux vidéo est visible depuis plusieurs postes clients.
- Sur le Pi, il suffit d'installer VLC :

```
apt-get install vlc
```

- Puis de rediriger la sortie de la commande **raspivid** vers VLC :

```
raspivid -t 0 -n --width 1280 --height 720 -o - | cvlc stream:///dev/stdin --
sout '#standard{access=http,mux=ts,dst=:8090}' :demux=h264
```

- Coté PC client, il suffit de lire le flux vidéo avec VLC grâce à la commande:

```
cvlc http://adresse_ip_du_pi:8090
```

Pour enregistrer le flux vidéo dans un fichier, utilisez la commande suivante:

```
cvlc http://adresse_ip_du_pi:8090 --sout file/ts:./Bureau/ma_video.h264
```

Faire du streaming

Méthode 3 : streaming de Pi à Pi avec OMXplayer

- Pour capturer un flux vidéo depuis un Pi pour l'observer depuis un autre Pi branché à un écran, il convient d'utiliser le lecteur **OMXplayer**.

Pour plus de détails voir:
http://www.magdiblog.fr/divers/raspberry-pi-camera-5-facons-de-faire-du-streaming/

Méthode 4 : mjpg-streamer

- Avec le **mjpeg**, on sort du streaming vidéo pur et dur, puisqu'il s'agit de transmettre et d'afficher une suite de photos les unes après les autres. Le résultat est beaucoup moins fluide

Méthode 5 : Serveur Web

Il n'existe pas de normes vidéo universellement prises en charge par tous les navigateurs Web. Cependant, pour les besoins, nous utiliserons un format beaucoup plus simple, le **MJPEG**. Le script suivant utilise le module **http.server** pour créer un simple serveur de streaming vidéo:

```
import io
import picamera
import logging
import socketserver
from threading import Condition
from http import server
PAGE="""\
<html>
<head>
<title>Raspberry Pi - Surveillance Camera</title>
</head>
<body>
<center><h1>Raspberry Pi - Surveillance
Camera</h1></center>
<center><img src="stream.mjpg" width="640"
height="480"></center>
</body>
</html>
"""
class StreamingOutput(object):
    def __init__(self):
        self.frame = None
        self.buffer = io.BytesIO()
        self.condition = Condition()
    def write(self, buf):
        if buf.startswith(b'\xff\xd8'):
  # New frame, copy the existing buffer's content
and notify all
            # clients it's available
            self.buffer.truncate()
            with self.condition:
                self.frame = self.buffer.getvalue()
                self.condition.notify_all()
            self.buffer.seek(0)
        return self.buffer.write(buf)class
StreamingHandler(server.BaseHTTPRequestHandler):
    def do_GET(self):
        if self.path == '/':
            self.send_response(301)
            self.send_header('Location', '/index.html')
            self.end_headers()
        elif self.path == '/index.html':
            content = PAGE.encode('utf-8')
            self.send_response(200)
            self.send_header('Content-Type', 'text/html')
            self.send_header('Content-Length', len(content))
            self.end_headers()
            self.wfile.write(content)
        elif self.path == '/stream.mjpg':
            self.send_response(200)
            self.send_header('Age', 0)
            self.send_header('Cache-Control', 'no-cache, private')
            self.send_header('Pragma', 'no-cache')
            self.send_header('Content-Type', 'multipart/x-mixed-
replace; boundary=FRAME')
            self.end_headers()
            try:
                while True:
                    with output.condition:
                        output.condition.wait()
                        frame = output.frame
                    self.wfile.write(b'--FRAME\r\n')
                    self.send_header('Content-Type', 'image/jpeg')
                    self.send_header('Content-Length', len(frame))
                    self.end_headers()
                    self.wfile.write(frame)
                    self.wfile.write(b'\r\n')
            except Exception as e:
                logging.warning(
                    'Removed streaming client %s: %s',
                    self.client_address, str(e))
        else:
            self.send_error(404)
            self.end_headers()

class StreamingServer(socketserver.ThreadingMixIn,
server.HTTPServer):
    allow_reuse_address = True
    daemon_threads = True

with picamera.PiCamera(resolution='640x480',
framerate=24) as camera:
    output = StreamingOutput()
    #Uncomment the next line to change your Pi's
Camera rotation (in degrees)
    #camera.rotation = 90
    camera.start_recording(output, format='mjpeg')
    try:
        address = ('', 8000)
        server = StreamingServer(address,
StreamingHandler)
        server.serve_forever()
    finally:
        camera.stop_recording()
```

Utilisation d'une webcam USB standard

- Plutôt que d'utiliser le module de caméra **Raspberry Pi**, on peut utiliser une **webcam USB** standard pour prendre des photos et des vidéos.
- Noter que la qualité du module de caméra est très supérieure à une webcam.
- Tout d'abord, installer le package *fswebcam*

```
sudo apt install fswebcam
```

- Si on n'utilise pas le compte utilisateur pi par défaut, on doit ajouter notre nom d'utilisateur au groupe vidéo, sinon on verra des erreurs «autorisation refusée».

```
sudo usermod -a -G video <username>
```

- Pour vérifier que l'utilisateur a été correctement ajouté au groupe, utilisez la commande *groups*.
- Entrer la commande *fswebcam* suivie d'un nom de fichier et une photo sera prise à l'aide de la webcam et enregistrée sous le nom de fichier spécifié:

```
fswebcam image.jpg
```

Utilisation d'une webcam USB standard

Deuxième méthode,

Il s'agit d'utiliser Putty

```
sudo apt-get update -y
lsusb     // vérifier la connection de la caméro
sudo apt-get install motion
sudo nano /etc/motion/motion.conf
sudo nano /etc/default/motion
```

Les modification à apporter dans le fichie **motion.config**
daemon on, ***width 640***, ***height 480***, ***framerate 10***, ***quality 100***, ***pre_capture 2***, ***post_capture 5***, ***stream_port 8081***, ***stream_quality 100***, ***stream_motion off*** et ***webcontrol_localhost off***.
Pour ecrire tapez ctr-o et entrer puis ctr-x pour sortir.
Un deuxième fichier à modifier qui est **motion**: ***start_motion_daemon=yes***
Une fois c'est fait, **motion** devient installé. Ensuite il faut reboute,
sudo reboot
Lancer le navigateur et taper Adr-ip:8081

Affichage des flux vidéo sur une page web

L'objectif est maintenant de transformer Raspberry Pi qui gère les caméras avec **motion** en un **serveur web** qui publie les flux vidéos.

Installation d'un serveur Web

Pour transformer le Pi en **serveur web** et mettre à notre disposition des informations issues de divers capteurs. La première étape est d'installer Apache.

Installer *Apache*

Apache2 est un serveur HTML des pages Web dynamiques en utilisant le PHP.

```
sudo apt install apache2 -y
```

Le lien ***http://adresse_IP_du_Pi*** permet d'accéder à la page Web *(/var/www/html/index.html*). Pour permettre au serveur Apache de traiter les fichiers PHP, il faut installer le module PHP et créer le fichier ***index.php***.

```
sudo apt install php libapache2-mod-php -y
```

Les flux vidéos d'une webcam sont capturés puis générés par **motion** et diffusés sur les ports 8081 et 8082 du Raspberry Pi.

Comment créer un serveur *MQTT* sur le Raspberry Pi

Principe de fonctionnement du MQTT

- *MQTT* est comme un *forum* avec des *abonnés* (lecteurs) et de *rédacteurs* qui eux *publient* des *messages*.
- Les *messages* sont hiérarchisés de la même manière, il est donc possible de consulter uniquement les *messages* qui nous intéressent.
- *MQTT* a été conçu pour économiser au maximum la batterie des objets.
  - Consomme **11 fois** moins d'énergie pour envoyer des messages,
  - Consomme **170 fois** moins pour en recevoir que le protocole HTTP.
  - Consomme **93 fois** plus rapide que le protocole HTTP.

MQTT et la QoS

- Il existe plusieurs niveaux de *qualité de service* avec *MQTT* Allant du:
  - Néant (*niveau 0*), un message est publié *sans garantie* de sa bonne réception,
  - Au niveau 2, appelé *service garantie*, qui se charge de publier le message en deux parties pour s'assurer et garantir, que le *message* est correctement délivré de façon unique.
- Plus on monte en niveau de *QoS*, plus on charge les trames.

Comment créer un serveur *MQTT* sur le Raspberry Pi

La sécurité et MQTT

- Du point de vue sécurité, si *MQTT* est parfaitement compatible avec des échanges à travers l'internet, il est possible de *tester* simplement le protocole sur des *brokers MQTT publiques*.
- On préfère des échanges sur un *réseau privé* avec un *serveur isolé* de l'extérieur. En effet, *MQTT* n'intègre pas de mécanisme de sécurité à proprement parlé.
- Il est possible de monter des *connexion SSL/TLS* soumises à *certificat*.
- Dans le cadre d'une *maison connectée*, il n'est pas utile de dialoguer avec l'extérieur en *MQTT*. On préférera un schéma plus classique visant à faire communiquer les objets en local avec le *serveur domotique*, qui se charge de relayer l'info à l'extérieur si nécessaire.

C'est donc tout l'intérêt de disposer d'un broker MQTT sur son réseau local à l'aide d'un Raspberry Pi.

Comment installer un broker Mosquitto sur le Raspberry Pi

- Il n'est pas nécessaire de disposer de la dernière version du Raspbery Pi.
- Si on a déjà un *Raspberry Pi* en fonctionnement pour une autre tâche, on peut lui attribuer cette tâche en plus.
- Pour ce faire, lançant la commande d'installation de ***Mosquitto***.

```
sudo apt-get install mosquitto
```

- Puis vérifier que tout est opérationnel avec la commande suivante.

```
systemctl status mosquitto
```

- C'est **terminé**, notre serveur *MQTT* local est prêt à collecter les messages !

- Pour faire des tests sur Raspberry, il faut installer les clients **mosquitto_sub**, **mosquitto_pub** et **mosquitto_passw** (pour simuler des utilisateurs),

```
sudo apt-get install mosquitto-clients
```

Comment installer un broker Mosquitto sur le Raspberry Pi

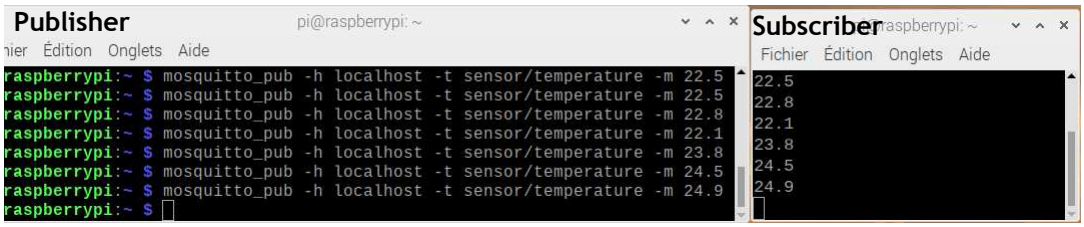
Envoyer/recevoir des messages MQTT

- Maintenant que le broker est prêt, dans un Terminal (***Subscriber***), exécuter la commande

```
mosquitto_sub -h localhost -t "sensor/temperature"
```

- Dans un autre Terminal (***Publisher***), publier notre premier message, on se connectant au ***serveur*** (-h) localhost sur le ***Topic*** (-t) sensor/temperature et on publie le **message** (-m) 24.9

```
mosquitto_pub -h localhost -t sensor/temperature -m 24.9
```



Utiliser MQTT avec Node-RED

- Il est simple de communiquer avec MQTT en utilisant Node-RED.
- Ajouter un node mqtt (section input).
- Double cliquer sur le ***node*** pour ouvrir la boîte de configuration. et appuyer sur le crayon pour ajouter une nouvelle connexion.
- Si ***Node-RED*** fonctionne sur la même machine que le broker Mosquitto, saisissez ***localhost*** dans le champ serveur et 1883 dans le port. Valider.

